

Lecture Notes: The Confusion Matrix, Precision, and Recall

Why accuracy alone lies — three worked examples, step by step, plus two exercises to solve by hand

Contents

1. Why Accuracy Is Not Enough	1
2. The Confusion Matrix	2
3. The Three Formulas	2
4. Worked Example 1 — A Spam Filter (compute everything, step by step)	3
5. Worked Example 2 — Disease Screening (where recall rules)	3
6. Worked Example 3 — Fraud Detection and the Accuracy Trap	4
7. When Precision Matters More, When Recall Matters More	5
8. Exercises — Solve by Hand	6
9. Summary	7

Concepts follow the BMC Machine Learning blog, “Confusion matrix in machine learning: Precision and recall explained” (Jonathan Johnson), <https://www.bmc.com/blogs/confusion-precision-recall/> — all numerical examples in these notes are original so every calculation can be verified by hand.

1. Why Accuracy Is Not Enough

You’ve built a classifier. It predicts at 86% accuracy on your test set. Sounds great — but that single number tells only half the story, and sometimes it gives the wrong impression altogether. A model can score high accuracy while being useless for the one thing you built it for (Example 3 will show this happening). To see the *whole* story we need the **confusion matrix**, and the two numbers read off it: **precision** and **recall**.

2. The Confusion Matrix

For a binary (yes/no) classifier, every prediction lands in exactly one of four cells, determined by two questions: what did the model *predict*, and what was *actually* true?

	Actually Positive	Actually Negative
Predicted Positive	TP — true positive	FP — false positive
Predicted Negative	FN — false negative	TN — true negative

The two diagonal cells (TP, TN) are the model being right. The two off-diagonal cells are the two *different ways of being wrong*, and telling them apart is the entire point:

- **False positive (FP)** — the model cried wolf: it predicted positive, but the truth was negative. (Predicting “spam” on a real email; flagging a healthy patient as sick.)
- **False negative (FN)** — the model missed: it predicted negative, but the truth was positive. (Letting spam into the inbox; telling a sick patient they’re healthy.)

These two mistakes almost never cost the same in the real world — and that asymmetry is what precision and recall measure.

3. The Three Formulas

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \quad \text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

Read them as questions:

- **Accuracy** — *Of everything, how much did I get right?* Right answers over all answers.
- **Precision** — *When I said “positive,” how often was I right?* Precision is punished by **false positives** — junk thrown into the positive pile. No FPs → 100% precision.
- **Recall** — *Of everything that was truly positive, how much did I find?* Recall is punished by **false negatives** — real positives that slipped through. No FNs → 100% recall.

Notice the two formulas share the numerator (TP) and differ only in which mistake sits in the denominator. Precision and recall are the yin and yang of the confusion matrix: each one forgives exactly the error that punishes the other. (A common companion, the **F1 score** = $2PR/(P+R)$, balances the two into one number.)

A memory trick: P and R both start from the positives. **P**recision starts from the **P**redicted positives (the model’s claims). **R**ecall starts from the **R**eal positives (the ground truth).

4. Worked Example 1 — A Spam Filter (compute everything, step by step)

A filter classifies 100 emails. In reality, 40 are spam (positive) and 60 are legitimate. The model's results:

	Actually spam (40)	Actually legit (60)
Predicted spam	TP = 30	FP = 5
Predicted legit	FN = 10	TN = 55

Step 1 — sanity-check the matrix. Rows and columns must add up: $30 + 5 + 10 + 55 = 100$ total (ok); actual spam column $30 + 10 = 40$ (ok); actual legit column $5 + 55 = 60$ (ok). Always do this first — most manual mistakes are caught right here.

Step 2 — accuracy.

$$\text{Accuracy} = \frac{TP + TN}{100} = \frac{30 + 55}{100} = \frac{85}{100} = \mathbf{0.85}$$

Step 3 — precision. The model *said* “spam” $TP + FP = 30 + 5 = 35$ times.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{30}{35} = \mathbf{0.857}$$

When this filter flags something, it's right 85.7% of the time — meaning 5 real emails (14.3% of its flags) were wrongly thrown in the spam folder.

Step 4 — recall. There *were* $TP + FN = 30 + 10 = 40$ actual spam emails.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{30}{40} = \mathbf{0.75}$$

The filter caught 75% of all spam; 10 spam emails leaked into the inbox.

Step 5 — interpret. Which mistake hurts more here? An FN means one annoying spam email in your inbox — you delete it and move on. An FP means a real message (a job offer, an invoice) silently buried in the spam folder. For a spam filter, **FPs are the expensive mistake, so precision is the metric to protect**, even at some cost in recall.

5. Worked Example 2 — Disease Screening (where recall rules)

A hospital screens 200 patients for a serious disease. In reality, 20 have it. The screening model produces:

	Actually sick (20)	Actually healthy (180)
Predicted sick	TP = 18	FP = 30
Predicted healthy	FN = 2	TN = 150

Step 1 — check the totals. $18 + 30 + 2 + 150 = 200$ (ok); sick column $18 + 2 = 20$ (ok).

Step 2 — accuracy. $(18 + 150)/200 = 168/200 = \mathbf{0.84}$.

Step 3 — precision. Flagged sick: $18 + 30 = 48$ people.

$$\text{Precision} = \frac{18}{48} = \mathbf{0.375}$$

Only 37.5% of the people it flags are actually sick. That looks terrible...

Step 4 — recall. Actually sick: $18 + 2 = 20$ people.

$$\text{Recall} = \frac{18}{20} = \mathbf{0.90}$$

...but it *found 90% of the sick patients*, missing only 2.

Step 5 — interpret the asymmetry. An FP here means a healthy person gets a follow-up test — some anxiety and cost, then relief. An FN means a sick person is sent home untreated, possibly to be diagnosed only when it's too late. The two mistakes are wildly unequal: **FNs are the catastrophic error, so recall is the metric to protect.** A screening tool with precision 0.375 and recall 0.90 is doing its job well; the follow-up test cleans up the false alarms. This is the same logic behind a content filter that blocks nudity on a photo-sharing platform: one banned image slipping through costs far more than over-flagging borderline images, so the classifier deliberately over-catches — trading precision for recall — because the cost of a miss is so high.

6. Worked Example 3 — Fraud Detection and the Accuracy Trap

A bank processes 1,000 transactions; only 10 are fraudulent. This is a heavily **imbalanced** dataset, and here accuracy actively deceives.

The “dumb” model. Predict *legitimate* for every single transaction, always:

$$TP = 0, \quad FP = 0, \quad FN = 10, \quad TN = 990 \quad \Rightarrow \quad \text{Accuracy} = \frac{0 + 990}{1000} = \mathbf{0.99}$$

Ninety-nine percent accuracy — from a model that has literally never detected anything! Its recall is $0/10 = \mathbf{0}$: it caught zero fraud. (Its precision is $0/0$ — undefined, since it

never made a positive claim at all.) This is the accuracy trap: **on imbalanced data, predicting the majority class gets a high accuracy score while completely failing the actual task.**

A real model. Now a genuine fraud detector on the same 1,000 transactions:

	Actually fraud (10)	Actually legit (990)
Predicted fraud	TP = 8	FP = 40
Predicted legit	FN = 2	TN = 950

Step by step: totals $8 + 40 + 2 + 950 = 1000$ (ok).

$$\text{Accuracy} = \frac{8 + 950}{1000} = \mathbf{0.958} \quad \text{Precision} = \frac{8}{8 + 40} = \frac{8}{48} = \mathbf{0.167} \quad \text{Recall} = \frac{8}{8 + 2} = \frac{8}{10} = \mathbf{0.8}$$

The punchline: the real model has *lower* accuracy than the dumb one (95.8% vs 99%) yet is enormously more valuable — it catches 8 of 10 frauds. Its 40 false positives mean 40 customers get a “was this you?” text message, a tiny cost next to the fraud it stops. Judged by accuracy alone you would deploy the useless model; judged by recall you deploy the right one.

7. When Precision Matters More, When Recall Matters More

The rule is always the same — **ask which mistake costs more**, an FP or an FN — then protect the metric that mistake punishes:

Cost structure	Protect	Scenarios
FP is expensive (a false alarm does real damage)	Precision	Spam filtering (burying a real email), criminal conviction (jailing an innocent person), auto-blocking user accounts, sending expensive marketing offers, auto-deleting “duplicate” files

Cost structure	Protect	Scenarios
FN is expensive (a miss does real damage)	Recall	Cancer/disease screening (sending a sick patient home), fraud detection (letting a theft through), airport security (missing a weapon), content moderation for prohibited images, fire/smoke alarms

Two tests that give the same answer: (1) *What happens to the victim of each error?* In screening, the FN victim may die; the FP victim takes an extra test. (2) *Is there a safety net after a positive prediction?* If a human reviews every flag (a doctor, a fraud analyst), false positives are cheap and you can afford to chase recall. If the positive prediction triggers an irreversible action automatically (delete, block, convict), false positives are expensive and precision rules.

Why not just maximize both? They pull against each other. Most classifiers output a score and apply a threshold: lower the threshold and you flag more things — recall rises (fewer misses) but precision falls (more junk in the flags); raise it and you flag only the sure things — precision rises but recall falls. You can picture the extremes: flag *everything* and recall is a perfect 1.0 while precision collapses to the base rate; flag only your single most confident case and precision is likely 1.0 while recall is nearly 0. A model can sit at an equilibrium where the two are equal, but squeezing a few extra points out of one will generally cost the other. Where to sit on that curve is not a math question — it's the cost/benefit question above.

8. Exercises — Solve by Hand

Work these with pencil and paper. Follow the same five steps as the worked examples: check totals → accuracy → precision → recall → interpret. Bring your worked answers to class, where we will go through them together.

Exercise A — Email spam filter. A filter processes 100 emails, of which 25 are actually spam. Its confusion matrix:

	Actually spam	Actually legit
Predicted spam	TP = 15	FP = 5
Predicted legit	FN = 10	TN = 70

1. Verify the totals.
2. Compute accuracy, precision, and recall.
3. Your colleague says “85% accuracy — ship it!” Using precision and recall, explain what accuracy hides. Which metric would *you* prioritize for a spam filter, and what does its value tell you here?

Exercise B — Airport baggage scanner. A scanner screens 500 bags; 20 actually contain a prohibited item. The model flags 63 bags in total, and 18 of the flagged bags truly contain a prohibited item.

1. Build the full confusion matrix from these three facts (find TP, FP, FN, TN).
 2. Compute accuracy, precision, and recall.
 3. Precision will come out low. Is this scanner bad? Which error type (FP or FN) is catastrophic here, which metric captures it, and what real-world process absorbs the cost of the other error type?
-

9. Summary

Accuracy counts right answers but treats all mistakes as equal; the confusion matrix splits mistakes into false positives and false negatives, which almost never cost the same. Precision = $TP/(TP+FP)$ asks “when I flag, am I right?” and is destroyed by false positives; recall = $TP/(TP+FN)$ asks “did I find them all?” and is destroyed by false negatives. Choose which to protect by asking which mistake is more expensive: irreversible actions on flagged items → precision; catastrophic misses → recall. On imbalanced data, accuracy can be almost meaningless — a do-nothing fraud model scored 99% — so always read precision and recall alongside it. And the two trade off through the decision threshold: you can buy more of one, but you pay in the other.